

操作系统第七周星期三讲稿整理

 **Lab2考试提醒：**今晚考Lab2，重点考察Q.H相关代码，特别是指针的指针操作，要融会贯通。

一、进程三态模型（复习+深入）

1.1 核心概念回顾

状态	含义	等待对象
就绪态 (Ready)	万事俱备，只欠CPU	等待CPU资源
执行态 (Running)	正在CPU上执行	占用CPU
阻塞态 (Waiting/Blocked)	等待外部条件满足	等待CPU以外的其他条件（如I/O完成、用户输入、信号量）

 **关键点：**

- 等待队列等待的不是CPU，而是**就绪条件**（外部条件）
- 当外部条件满足后，进程从阻塞→就绪，然后还需要在就绪队列排队等CPU调度
- 不是一唤醒就直接上CPU执行

1.2 状态转换的深入理解

关键判断题解析：

1.  "没有Running进程就一定没有Ready进程"
- 错误。所有进程可能都在阻塞等待外部条件（如都在等用户输入），此时既没有Running也没有Ready
2.  "如果没有Ready进程，系统一定没有Running进程"
- 正确。因为CPU不会空转，如果有可运行的进程一定会调度上去
3.  "不同的进程必然对应不同的程序"
- 错误。同一个程序（如Word）可以打开多个进程实例
4.  "不同的程序必须运行在不同的进程中"
- 错误。一个进程中可以执行多个程序（如Shell进程fork后调用execv执行新程序）

1.3 fork与execv的区别

系统调用	作用	代码段变化
fork()	创建新进程，子进程复制父进程上下文	父子进程共享相同的代码段（还是原来的程序）

系统调用	作用	代码段变化
execv()	装载执行新程序	替换当前进程的代码段、数据段，整个进程的魂换了

💡 **理解**：fork是"分身"，execv是"换魂"。Shell的工作流程就是：fork出子进程，然后子进程调用execv去执行真正的用户命令。

二、PCB与进程控制的深入理解

2.1 PCB的访问权限

✗ "用户进程可以自行修改PCB"

- **PCB是操作系统内核维护的数据结构**，用户进程绝对不能直接修改
- 用户想修改PCB相关内容（如申请资源、提权）必须通过**系统调用**
- 如果用户能直接改PCB，系统安全就彻底崩溃了（比如直接把自己改成root权限）

2.2 进程能否主动修改自身状态

状态转换	能否主动?	说明
Running → Waiting	✅ 可以主动	通过系统调用（如等待I/O、等待信号量）主动进入阻塞
Running → Ready	✅ 可以主动	通过yield系统调用主动让出CPU，回到就绪队列
Ready → Running	✗ 不能主动	只能被动等待调度器选中
Waiting → Ready	✗ 不能主动	只能等待外部事件唤醒（如中断）

💡 **yield系统调用**：主动让出CPU，回到就绪队列排队。这是主动的、友善的行为。

2.3 父子进程资源继承

- **相同/继承的资源**：打开的文件描述符、内存空间（写实复制）、各种权限
- **必然不同的资源**：PID、父进程ID、页表（虽然内容初始相同，但物理上是独立的）
- **写实复制 (Copy-on-Write)**：fork后父子共享页表，只有当某一方尝试写时才真正复制页表

2.4 中断与进程切换的关系

✗ "中断是进程切换的充分必要条件"

关系	说明
中断 → 切换?	✗ 不一定。中断发生后，可能处理完中断直接返回原进程，不切换
切换 → 中断?	✗ 不一定。进程主动调用yield也会触发切换，不一定需要外部中断

💡 中断和进程切换没有必然的因果关系，中断只是可能触发调度的时机之一。

三、进程同步与互斥基础

3.1 为什么需要同步互斥？

进程的三大特征：**并发、共享、不确定性**

- 并发导致多个进程同时访问共享数据
- 如果不加控制，执行结果依赖于访问的特定次序 → 产生竞争条件（Race Condition）

 **竞争条件定义：**多个进程并发访问和操作同一个数据，且执行结果与访问的特定顺序有关，称为发生了竞争。

竞争产生的条件：

- 多个进程都对共享数据进行**写操作**（只读不会有问题）
- 操作不是原子的，中间可以被打断

3.2 临界资源与临界区

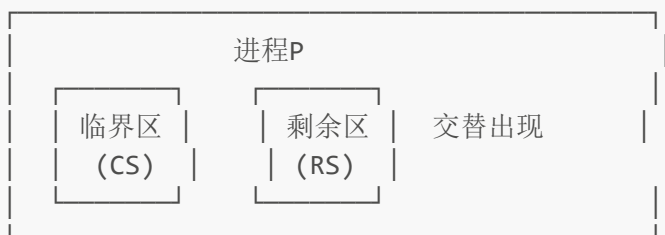
概念	定义
临界资源（Critical Resource）	一次只允许一个进程访问的资源
临界区（Critical Section）	每个进程中访问临界资源的那段代码

 **例子：**

- 打印机是临界资源（不能同时打印两份文档，否则会乱码）
- 打印队列是临界资源（多个进程同时写队列会导致任务丢失）
- 进程中“往打印队列写任务”的那段代码就是临界区

3.3 程序结构划分

一个进程的代码可以划分为：



- **临界区（Critical Section）：**一次只允许一个进程进入
- **剩余区（Remainder Section）：**其他不访问临界资源的代码，可以并发执行

3.4 临界区访问的状态变化

A想进入临界区 → A在临界区执行 → A退出临界区
↓
B想进入时如果A还在里面 → B被阻塞（wait）

💡 阻塞队列等待的不一定是硬件中断，也可以是程序中定义的临界区访问条件。

四、临界区管理的准则 ★★ ★

🔴 十六字箴言：空闲让进、忙则等待、有限等待、让权等待

4.1 四条准则详解

准则	含义	解决的问题
空闲让进	临界区空闲时，想进的进程应该能立即进入	保证资源利用率，避免资源浪费
忙则等待	已有进程在临界区时，其他想进的必须等待	保证互斥，这是最基本的要求
有限等待	任何进程进入临界区的要求必须在有限时间内得到满足	保证公平性，防止“饥饿”（死等）
让权等待	当进程不能进入临界区时，应主动释放CPU，避免忙等	提高CPU效率，不让CPU空转

4.2 逐条深入理解

1. 空闲让进（Mutual Exclusion其实包含在忙则等待里）

- 临界区空着，就不要设置额外门槛不让人进
- 这是最基本的：没人用的时候应该让想用的人用

2. 忙则等待（互斥的核心）

- 这是临界区的定义本身要求的：一次只能一个进
- 有人在里面的时候，其他人必须在外面等

3. 有限等待（防止饥饿）

🎯 这是考点重点

- 什么是饥饿（Starvation）：一个进程想进临界区，但一直被其他进程插队，永远轮不到它
- 有限等待就是防止饥饿：必须保证等待时间是有上界的，不能无限等下去
- 反面例子：有人拿书包占座，或者有人永远插队，导致你永远坐不上

4. 让权等待（提高效率）

- 什么是忙等（Busy Waiting）：进不去临界区的时候，CPU不空转循环检查“我能进了吗？能进了吗？”
- 让权等待：发现进不去的时候，主动把CPU让出来，自己去阻塞队列睡觉
- 这主要是效率问题，不是正确性问题，但如果不这样做CPU利用率会很低

💡 考试提示：这四条准则经常考选择题，特别是“有限等待”和“让权等待”的区别要搞清楚——一个是防止永远等不到，一个是防止等的时候浪费CPU。

五、互斥与同步的对比理解

5.1 两个核心概念

关系	特点	典型场景
互斥 (Mutex)	进程间 间接制约 关系。我在临界区的时候你不能进，但我不知道我排斥的具体是谁，只是"有人在我就不能进"。是被动的排斥关系。	打印机、共享变量的访问
同步 (Sync)	进程间 直接制约 关系。我知道其他进程的存在，主动跟它们协调，刻意安排执行的先后顺序。是主动的合作关系。	生产者-消费者（你生产了我才能消费）

5.2 关键区别

维度	互斥	同步
关系性质	间接制约、被动排斥	直接制约、主动协作
是否知道对方	不知道具体排斥谁	明确知道协作对象是谁
有无顺序要求	无严格顺序，谁先进都行，只要不同时进	有严格的执行先后顺序，是刻意安排的
解决目标	保证临界资源的安全访问	保证多个进程协作的逻辑正确性

 **老师原话：**

- 互斥是"你先干完我再干"，但不知道是谁
- 同步是"你先干完我再干"，而且明确知道是你，是刻意安排的先后顺序

六、打印队列竞争的例子（理解竞争条件）

问题场景

多个Word进程同时往打印队列里写任务：

```
// 每个Word进程执行这段代码：
tail = read_print_queue_tail(); // 读当前队尾位置
write_task_to_queue(tail, my_task); // 写我的任务
tail = tail + 1; // 队尾后移
```

竞争发生的时序：

时间	进程A	进程B	共享变量tail
T1	读tail=3		3
T2	被切换出去		3
T3		读tail=3	3
T4		往位置3写任务B	3
T5	被调度回来		3

时间	进程A	进程B	共享变量tail
T6	往位置3写任务A		3（覆盖了！）
T7	tail=4		4
T8		tail=4	4

结果：两个进程都写到同一个位置，B的任务被A覆盖丢失了！

解决方案

把"读tail→写队列→改tail"这整个三段代码变成**临界区**：

```

进入临界区();    // 一次只许一个进
    tail = read_print_queue_tail();
    write_task_to_queue(tail, my_task);
    tail = tail + 1;
退出临界区();    // 出来后下一个才能进

```

这样就保证了整个操作是原子的，不会被打断。

本章考点总结

考点	考察形式	重要程度
进程三态转换（不存在的边）	选择题、判断题	☆☆☆
临界区管理四条准则（十六字）	选择题、简答题	☆☆☆
互斥与同步的区别	简答题、选择题	☆☆
竞争条件产生的原因	简答题	☆☆
fork与execv的区别	选择题	☆☆
用户能否直接修改PCB	判断题	☆